

Oct 05, 22 18:25

main.py

Page 1/2

```
#!/bin/env python3.8

"""
Michael Bentivegna
Deep Learning Assignment 5
Professor Curro

Data was retrieved from the same source as here (see lines 52 and 53):
https://huggingface.co/datasets/ag_news/blob/main/ag_news.py
"""

import os
import logging

import pandas as pd
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers.core import Dense, Flatten, Dropout
from keras_preprocessing.text import Tokenizer
from keras_preprocessing.sequence import pad_sequences
from keras.layers import Embedding
from absl import app
from absl import flags

script_path = os.path.dirname(os.path.realpath(__file__))

# Define default values for flags
FLAGS = flags.FLAGS
flags.DEFINE_integer("batch_size", 32, "Number of samples in batch")
flags.DEFINE_integer("num_epochs", 3, "Number of SGD iterations")
flags.DEFINE_bool("debug", False, "Set logging level to debug")

def getData():
    """
    Get text data for ag news and return train, validation, and test
    """
    train = pd.read_csv(
        script_path + "/ag_news_train.csv", names=["label", "headline", "description"]
    )
    test = pd.read_csv(
        script_path + "/ag_news_test.csv", names=["label", "headline", "description"]
    )

    # Combine headline and description into one string
    # Subtract 1 from labels so 1-4 becomes 0-3
    x_all_train = train["headline"] + train["description"]
    y_all_train = train["label"] - 1

    x_test = test["headline"] + test["description"]
    y_test = test["label"] - 1

    x_train, x_val, y_train, y_val = train_test_split(
        x_all_train,
        y_all_train,
        test_size=0.1,
        shuffle=True,
    )

    return x_train, x_val, x_test, y_train, y_val, y_test
```

Oct 05, 22 18:25

main.py

Page 2/2

```
def main(a):
    logging.basicConfig()

    if FLAGS.debug:
        logging.getLogger().setLevel(logging.DEBUG)

    # Get Data
    x_train, x_val, x_test, y_train, y_val, y_test = getData()

    # Tokenize the headline / description combination
    t = Tokenizer()
    t.fit_on_texts(x_train)
    e_x_train = t.texts_to_sequences(x_train)
    e_x_val = t.texts_to_sequences(x_val)
    e_x_test = t.texts_to_sequences(x_test)

    # Add padding to ensure each array is the same size in training set
    p_x_train = pad_sequences(e_x_train, padding="post")

    # Minimize padding by using max sequence length of training set (potential cutoffs)
    str_max = p_x_train.shape[1]
    p_x_val = pad_sequences(e_x_val, padding="post", maxlen=str_max)
    p_x_test = pad_sequences(e_x_test, padding="post", maxlen=str_max)

    # Get number of unique tokens (keras docs said to add 1)
    vocab_size = len(t.word_counts) + 1

    # Build Sequential Model with proper embedding
    model = Sequential()
    model.add(Embedding(vocab_size, int(str_max * (1 / 4)), input_length=str_max))

    model.add(Flatten())
    model.add(Dense(128, activation="relu", kernel_regularizer="l2"))
    model.add(Dropout(0.1))
    model.add(Dense(4, activation="softmax"))

    model.compile(
        loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"]
    )

    # Train model
    model.fit(
        p_x_train,
        y_train,
        validation_data=(p_x_val, y_val),
        batch_size=FLAGS.batch_size,
        epochs=FLAGS.num_epochs,
        verbose=1,
    )

    # Test model
    print("-----Evaluate Classification Model on Test Set-----")
    model.evaluate(p_x_test, y_test, verbose=1)

if __name__ == "__main__":
    app.run(main)
```